DEVOPS · KUBERNETES · MICROSERVICES

VProfile on Kubernetes

Deploying a multi-tier Java application as 4 microservices on a real HA cluster — we build the images ourselves, harden them, and wire them together step by step. A complete, teaching-first guide.

6	RHEL 9	4
nodes · 3 masters + 3 workers	masters & workers	custom secure images

Docker Hub: alnaqib · GitHub: yousefsalemW
HA cluster fronted by HAProxy · k8s v1.36

ALnaqib

TABLE OF CONTENTS

Contents

The guide walks through one microservice at a time. Each part has: the hardened Dockerfile, the build commands, the k8s manifests, and most importantly — the **"why"** behind every decision.

THE BIG PICTURE

Overview & architecture

00

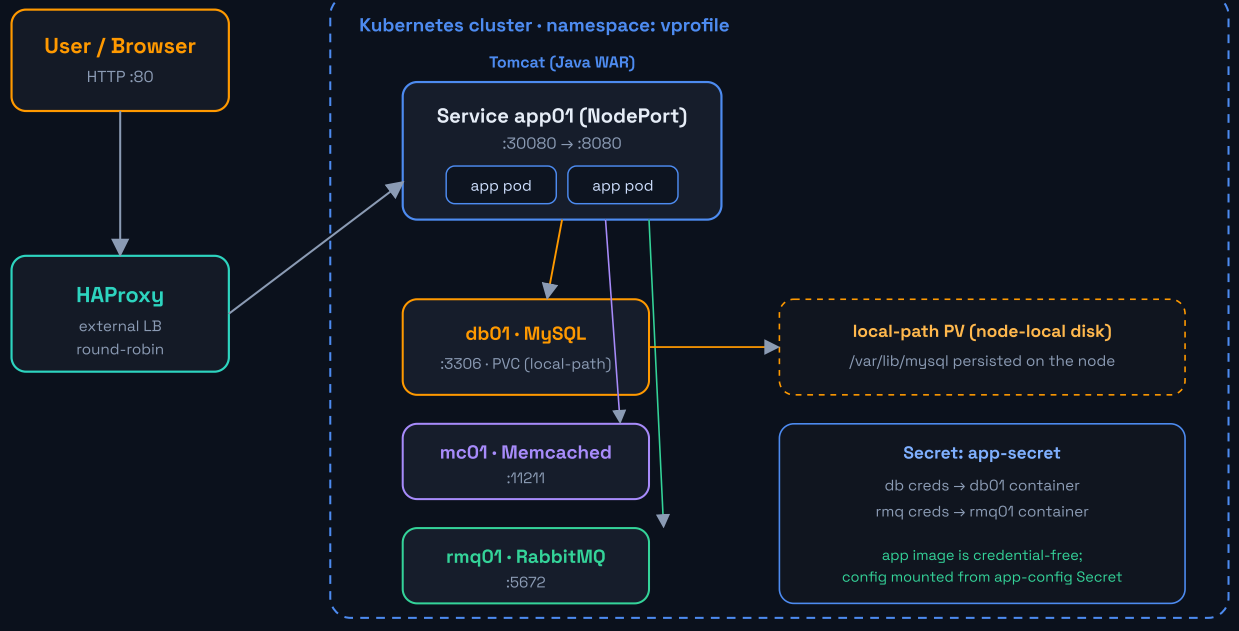
VProfile is a classic Java Spring MVC application made of several tiers working together: a web front, the app, a database, a cache, and a message broker. Originally each tier was its own VM (Vagrant). We re-platform it into microservices on the cluster.

The map: from VM to Kubernetes

Old tier (VM)	Hostname	New shape on k8s	Service	Port
Nginx (web)	web01	dropped — HAProxy takes its role	–	80
Tomcat (app)	app01	Deployment (2 replicas) + NodePort	app01	8080
MySQL	db01	Deployment + PVC + ClusterIP	db01	3306
Memcached	mc01	Deployment + ClusterIP	mc01	11211
RabbitMQ	rmq01	Deployment + ClusterIP	rmq01	5672

WHY

The app already reads its service addresses from `application.properties` using names like `db01`, `mc01`, `rmq01`. That is exactly how k8s DNS works: a Service named `db01` inside a namespace called `vprofile` resolves to `db01.vprofile.svc.cluster.local`. So service discovery works with almost no code change — which is why the Service names must be exactly `db01/mc01/rmq01`.



traffic: User → HAProxy → NodePort(app01) → app pods → db01 / mc01 / rmq01

NOTE

There is ElasticSearch config in the code, but no service or script for it in the original project, so it is **optional** and we skip it. The app runs fine without it.

TIP

We dropped Nginx (web01) because the existing **HAProxy** plays exactly that role (reverse proxy + load balancing) at the cluster edge — no need for an extra tier inside k8s.

Who runs where · node placement

Important: we do **not** manually decide whether a pod lands on a master or a worker — that is the scheduler's job. But you must understand the default outcome, and separate **"where a thing runs"** from **"where you run a command"**.

Component	Runs on	Decided by
Control-plane (etcd · apiserver · scheduler · controller-mgr)	3 masters	`kubeadm`
kube-proxy + CNI (DaemonSet)	all 6 nodes	`tolerates taint`
db01 pod + PVC	one worker (pinned)	`scheduler + volume affinity`
mc01 / rmq01 pods	on the workers	`scheduler`
app pods (×2)	spread across workers	`scheduler`
HAProxy	its own VM, outside the cluster	`you`
build & push images	your workstation	`you`
`kubectl apply` / deploy.sh	from a master (has kubeconfig)	`you`

WHY

In a `kubeadm` cluster the masters carry a taint named `node-role.kubernetes.io/control-plane:NoSchedule`. It means a normal workload pod will **not** schedule on a master unless it has a toleration. So as long as you did not remove the taint — **all VProfile pods (db/mc/rmq/app) land on the workers**, and the masters stay for the control plane.

NOTE

Exception: `kube-proxy` and the CNI run as a DaemonSet on **every** node (masters too) because they tolerate the taint. So the masters are not empty — they run the control plane plus these DaemonSets.

GOTCHA

`db01` in particular gets pinned to one worker automatically: because the local-path PV is node-local, the provisioner adds `nodeAffinity` to the PV, so the pod returns to the node that holds its data (more in part 03).

Check the actual placement by looking at the NODE column:

verify placement — BASH

```
$ kubectl get pods -n vprofile -o wide   # the NODE column shows which node each pod landed on
```

TIP

If you **want** to force a pod onto a specific node (rarely needed): use `nodeSelector`, `nodeAffinity`, or `tolerations` (to run on a master on purpose). But the advice is: let the scheduler do its job, do not fight it without a real reason.

SECURE BY DESIGN

Image security philosophy

01

Before any Dockerfile, let us agree on the principles we follow for **every** image. These are not cosmetic — they are the difference between a production-ready image and a liability.

Principle	How we apply it
Multi-stage build	Build tools (Maven/JDK) live in their own stage; the runtime carries only the `.war` — smaller image, smaller attack surface.
Non-root user	Every image runs as an ordinary user, not `root` (tomcat / mysql / memcache / rabbitmq).
Pinned versions	No `:latest` — every base image is pinned to a specific tag for reproducibility.
No secrets in layers	The app image is fully **credential-free**; any password is stripped at build, and the real config is injected at runtime from a Secret (mount).
Minimal base	`alpine` for the cache and broker — smaller and cleaner.
Scan (Trivy)	Every image is scanned for `HIGH/CRITICAL` vulns before we push it.

WHY

”Security” is not a final step you bolt on — it is a decision on every line: which base image, which user, where the secrets live, what goes into the runtime. Applying these principles from the start builds a clean image instead of patching a bad one.

NOTE

All nodes are **RHEL 9 / amd64**, so we build with `--platform linux/amd64` to avoid architecture mismatches if you build from a different machine.

BEFORE WE START

Prerequisites

02

We need to confirm the cluster is ready, and that the build tools are on the workstation.

1) Confirm the cluster is healthy

on master1 — BASH

```
$ kubectl get nodes
NAME      STATUS   ROLES           VERSION
master1   Ready    control-plane   v1.36.2
master2   Ready    control-plane   v1.36.2
master3   Ready    control-plane   v1.36.2
worker1   Ready    worker          v1.36.2
worker2   Ready    worker          v1.36.2
worker3   Ready    worker          v1.36.2
```

2) Build tools on the workstation

On the build machine (your laptop, say) you need: `git`, `docker` (or `podman` on RHEL), and `trivy` for scanning.

workstation — BASH

```
# docker installed?
$ docker version

# trivy (scanner) — on RHEL/Fedora
$ sudo dnf install -y trivy   # or the official binary

# get the source (app + db images build from it)
$ git clone -b Master https://github.com/abdelrahmanonline4/sourcecodeseniorwr.git src
```

NOTE

The nodes run containers with `containerd` (not docker) — that is unrelated to building images. You build the images on your machine with docker/podman and push to Docker Hub; the nodes pull from there. This separation is intentional.

3) The namespace

We isolate the whole project in a namespace called `vprofile` (the same name the code expects in DNS).

k8s/00-namespace.yaml — YAML

```
apiVersion: v1
kind: Namespace
metadata:
  name: vprofile
  labels:
    app.kubernetes.io/part-of: vprofile
```

WHY

Every resource (Deployments/Services/Secrets/PVC) goes inside this namespace. It gives clean isolation and makes internal DNS resolve the way the app expects: `db01.vprofile` becomes `db01.vprofile.svc.cluster.local`.

STATE NEEDS A HOME

Storage: local-path-provisioner

03

The database needs persistent storage — if the pod is removed, the data must survive. A hand-built `kubeadm` cluster usually has no default StorageClass, so a `PVC` would stay `Pending` forever. The fix: install **local-path-provisioner**.

Install (once per cluster)

on master1 — BASH

```
# install the provisioner (Rancher)
$ kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.31/deploy/local-path-storage.yaml

# wait for its pod to be ready
$ kubectl -n local-path-storage rollout status deploy/local-path-provisioner

# make it the default StorageClass
$ kubectl patch storageclass local-path \
    -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'

$ kubectl get sc
NAME                  PROVISIONER             DEFAULT
local-path (default)  rancher.io/local-path   true
```

WHY

This provisioner automatically creates a PV for each PVC on the node local disk (under `/opt/local-path-provisioner`). Simple, lightweight, and a great fit for a teaching/internal cluster without a SAN or cloud storage.

GOTCHA

Because storage is **node-local**, the PV is bound to the node it was created on, and the provisioner adds `nodeAffinity` automatically so the pod keeps scheduling on the same node. Enough for a single-replica DB. For a truly highly-available DB you would need networked storage (Longhorn / Ceph / NFS).

MICROSERVICE · DATABASE

Database · db01

04

Role: the `accounts` database holding users and roles, seeded from `db_backup.sql`. We use MySQL 8.0 — what the app was designed for (the connector in the `pom` is version 8.0.22).

1) The Dockerfile (hardened)

images/db/Dockerfile — DOCKERFILE

```
# ========================================================
#  VProfile DB (MySQL 8)  -  pre-seeded 'accounts' database
#  build context = ../../src   (the source-code repo root)
# ========================================================
FROM mysql:8.0.40
LABEL maintainer="ALnaqib" \
      org.opencontainers.image.title="vprofile-db"

# database name is fixed; ALL passwords come at runtime from a k8s Secret
ENV MYSQL_DATABASE=accounts

# any *.sql here runs ONCE on first boot (empty data dir), in file order
# (db_backup.sql is a MySQL 5.7 dump; imports cleanly into MySQL 8.0)
COPY src/main/resources/db_backup.sql /docker-entrypoint-initdb.d/10-db_backup.sql

EXPOSE 3306
# NOTE: the official mysql image already runs as the unprivileged 'mysql' user
```

WHY

Two key things: (1) `db_backup.sql` goes into `/docker-entrypoint-initdb.d/` — any file there runs **once** on first boot (empty data dir), so the tables are created automatically. (2) No passwords live in the image — they arrive at runtime from a Secret via the `MYSQL_*` environment variables.

2) The Secret (single source for passwords)

k8s/01-secret.yaml — YAML

```
# ------------------------------------------------------------------
#  One Secret feeds the DB + RabbitMQ containers at runtime.
#  IMPORTANT: db-user/db-password/rmq-user/rmq-password MUST match the
#  --build-arg values you baked into the APP image, otherwise the app
#  will authenticate with the wrong credentials.
#  (stringData = plain text now, kubernetes base64-encodes it for you)
# ------------------------------------------------------------------
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
  namespace: vprofile
type: Opaque
stringData:
  db-root-password: "ChangeMe_Root_123"
  db-user: "admin"
  db-password: "admin123"
  rmq-user: "vprofile"
  rmq-password: "guest"
```

GOTCHA

These values must **match** the ones in the `app-config` Secret (which the app reads its config from — see part 07). If they differ, the app authenticates with the wrong password and fails.

3) PVC + Deployment + Service

k8s/02-db.yaml — YAML

```yaml
# ================= PersistentVolumeClaim =================
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-pvc
  namespace: vprofile
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: local-path      # provided by local-path-provisioner
  resources:
    requests:
      storage: 3Gi
---
# ================= Deployment =================
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vpro-db
  namespace: vprofile
spec:
  replicas: 1                       # a single DB pod owns the volume (RWO)
  selector:
    matchLabels: { app: vpro-db }
  strategy:
    type: Recreate                  # never run 2 pods on one RWO volume
  template:
    metadata:
      labels: { app: vpro-db }
    spec:
      containers:
        - name: vpro-db
          image: alnaqib/vprofile-db:1.0
          # force native password ⇒ old JDBC connects over plain TCP with no
          # caching_sha2 / public-key retrieval issues
          args: ["--default-authentication-plugin=mysql_native_password"]
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom: { secretKeyRef: { name: app-secret, key: db-root-password } }
            - name: MYSQL_USER
              valueFrom: { secretKeyRef: { name: app-secret, key: db-user } }
            - name: MYSQL_PASSWORD
              valueFrom: { secretKeyRef: { name: app-secret, key: db-password } }
          volumeMounts:
            - name: db-data
              mountPath: /var/lib/mysql
      volumes:
        - name: db-data
          persistentVolumeClaim:
            claimName: db-pvc
---
# ================= Service (name MUST be db01) =================
apiVersion: v1
kind: Service
metadata:
  name: db01                        # app talks to db01.vprofile:3306
  namespace: vprofile
spec:
```

```
selector: { app: vpro-db }
ports:
  - port: 3306
    targetPort: 3306
# ClusterIP (default) ⇒ internal only
```

WHY

strategy: Recreate and **replicas: 1** are deliberate: the volume is `ReadWriteOnce`, so only one pod can hold it. With the default `RollingUpdate`, k8s would start a new pod before killing the old one, and the two would fight over the same volume. `Recreate` stops the old pod first.

GOTCHA

Note the `args: ["--default-authentication-plugin=mysql_native_password"]` in the Deployment. MySQL 8 defaults users to `caching_sha2_password`, which over a non-TLS connection (like inside the cluster) sometimes throws `Public Key Retrieval is not allowed` with JDBC. Forcing `mysql_native_password` lets the `admin` user connect over plain TCP with no pain. (This option exists in MySQL 8.0; in 8.4 it was removed in favor of `--mysql-native-password=ON`).

NOTE

`db_backup.sql` is a MySQL 5.7 dump — it imports into MySQL 8.0 fine; at most you see a harmless charset warning (`utf8` → `utf8mb3`).

MICROSERVICE · CACHE

Cache · mc01

05

Role: Memcached stores frequent query results in RAM to take load off the database. No durable state — if it is gone, it is rebuilt from the DB.

1) The Dockerfile

images/memcached/Dockerfile — DOCKERFILE

```
# ============================================================
#  VProfile CACHE (Memcached)  —  tiny hardened image
#  build context = .  (this folder)
# ============================================================
FROM memcached:1.6.31-alpine
LABEL maintainer="ALnaqib" \
      org.opencontainers.image.title="vprofile-mc"

EXPOSE 11211
# -m 64  : cap RAM at 64 MB   |  -l 0.0.0.0 : listen on the pod network
# the base image already runs as the unprivileged 'memcache' user
CMD ["memcached", "-m", "64", "-l", "0.0.0.0", "-p", "11211"]
```

WHY

`-l 0.0.0.0` is required: by default memcached listens only on `127.0.0.1`, which means no other pod can reach it. In k8s the connection comes from the pod network, so it must listen on all interfaces. And `-m 64` caps RAM at 64MB so it does not eat the node resources.

2) Deployment + Service

k8s/03-memcached.yaml — YAML

```
# ================= Deployment =================
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vpro-mc
  namespace: vprofile
spec:
  replicas: 1
  selector:
    matchLabels: { app: vpro-mc }
  template:
    metadata:
      labels: { app: vpro-mc }
    spec:
      containers:
        - name: vpro-mc
          image: alnaqib/vprofile-mc:1.0
          ports:
            - containerPort: 11211
---
# ================ Service (name MUST be mc01) ================
apiVersion: v1
kind: Service
metadata:
  name: mc01                        # app talks to mc01.vprofile:11211
  namespace: vprofile
```

```
spec:
  selector: { app: vpro-mc }
  ports:
    - port: 11211
      targetPort: 11211
```

NOTE

No PVC here — it is a cache, and a cache is meant to be lost and rebuilt. Adding persistence would over-complicate something that should stay simple.

MICROSERVICE · BROKER

Message broker · rmq01

06

Role: RabbitMQ receives messages from the app and delivers them (producer/consumer pattern). The app connects to it on `5672`.

1) The Dockerfile

images/rabbitmq/Dockerfile — DOCKERFILE

```
# ============================================================
#  VProfile BROKER (RabbitMQ)  —  management + hardened base
#  build context = .  (this folder)
# ============================================================
FROM rabbitmq:3.13.7-management-alpine
LABEL maintainer="ALnaqib" \
      org.opencontainers.image.title="vprofile-rmq"

# 5672  = AMQP (used by the app)   |   15672 = management UI
EXPOSE 5672 15672
# the login user/pass are injected at runtime via RABBITMQ_DEFAULT_USER/PASS
# (a real, non-'guest' user ⇒ works over the network with no loopback tricks)
```

WHY

The original user was `guest`, but RabbitMQ blocks `guest` from connecting from anywhere but localhost for security. Instead of lifting that restriction (a bad idea), we create a real user via `RABBITMQ_DEFAULT_USER/PASS` from the Secret — so it works over the network cleanly and safely.

2) Deployment + Service

k8s/04-rabbitmq.yaml — YAML

```
# ================= Deployment =================
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vpro-rmq
  namespace: vprofile
spec:
  replicas: 1
  selector:
    matchLabels: { app: vpro-rmq }
  template:
    metadata:
      labels: { app: vpro-rmq }
    spec:
      containers:
        - name: vpro-rmq
          image: alnaqib/vprofile-rmq:1.0
          ports:
            - containerPort: 5672      # AMQP
            - containerPort: 15672     # management UI
          env:
            - name: RABBITMQ_DEFAULT_USER
              valueFrom: { secretKeyRef: { name: app-secret, key: rmq-user } }
            - name: RABBITMQ_DEFAULT_PASS
              valueFrom: { secretKeyRef: { name: app-secret, key: rmq-password } }
---
```

```
# ================= Service (name MUST be rmq01) =================
apiVersion: v1
kind: Service
metadata:
  name: rmq01                       # app talks to rmq01.vprofile:5672
  namespace: vprofile
spec:
  selector: { app: vpro-rmq }
  ports:
    - port: 5672
      targetPort: 5672
```

GOTCHA

The rmq user/password here must also match the values in the `app-config` Secret (rabbitmq.username / rabbitmq.password).

MICROSERVICE · APPLICATION

Application · app01

07

This is the heart of the project: Tomcat 9 running `vprofile-v2.war`. This image matters most for security, so we made it fully **credential-free — no password inside it at all.**

1) Multi-stage Dockerfile (secret-free)

images/app/Dockerfile — DOCKERFILE

```
# ============================================================
#  VProfile APP (Tomcat)  -  multi-stage, CREDENTIAL-FREE image
#  build context = ../../src   (the source-code repo root)
#  NO passwords live in this image. The real application.properties
#  is mounted at runtime from a k8s Secret (see k8s/05-app.yaml).
# ============================================================

# ---------- Stage 1: build + explode the WAR ----------
FROM maven:3.9.9-eclipse-temurin-11 AS build
WORKDIR /workspace
COPY pom.xml .
COPY src ./src

# strip any credential from the properties that get compiled into the WAR,
# so nothing sensitive ever lands in an image layer
RUN set -eux; p=src/main/resources/application.properties; \
    sed -i "s#^jdbc.password=.*#jdbc.password=__SET_AT_RUNTIME__#"         "$p"; \
    sed -i "s#^rabbitmq.password=.*#rabbitmq.password=__SET_AT_RUNTIME__#" "$p"; \
    mvn -q -B clean package -DskipTests; \
    mkdir -p /workspace/target/exploded && cd /workspace/target/exploded && \
    jar -xf ../vprofile-v2.war

# ---------- Stage 2: minimal runtime ----------
FROM tomcat:9.0.98-jre11-temurin-jammy
LABEL maintainer="ALnaqib" \
      org.opencontainers.image.title="vprofile-app" \
      org.opencontainers.image.source="https://github.com/yousefsalemW"

RUN rm -rf /usr/local/tomcat/webapps/*
# ship the webapp EXPLODED so a Secret can be mounted over its properties file
COPY --from=build /workspace/target/exploded/ /usr/local/tomcat/webapps/ROOT/

# run as an unprivileged user (hardening)
RUN groupadd -r tomcat && useradd -r -g tomcat -d /usr/local/tomcat tomcat \
    && chown -R tomcat:tomcat /usr/local/tomcat
USER tomcat

EXPOSE 8080
CMD ["catalina.sh", "run"]
```

WHY

Stage one (Maven) does two things: (1) it strips any password from `application.properties` before building (replacing it with `__SET_AT_RUNTIME__`) — so no secret enters any image layer. (2) After building the `.war`, it explodes it into a folder — so we can mount the real config file over it at runtime. Stage two (Tomcat) takes only the exploded folder, with no Maven or JDK — **a small, safe, secret-free image.**

2) Real config: mounted from a Secret at runtime

The file baked into the image has no passwords. The real values (hosts + credentials) live in a Secret named `app-config`, which we mount exactly over `application.properties` inside the webapp — so the app reads it naturally, as if it were built in.

k8s/05-app.yaml YAML

```
# ============================================================
#  app-config: the FULL application.properties, kept OUT of the
#  image and injected at runtime. This is the app's single
#  source of truth for hosts + credentials.
#  (put strong, real passwords here; they must match 01-secret.yaml)
# ============================================================
apiVersion: v1
kind: Secret
metadata:
  name: app-config
  namespace: vprofile
type: Opaque
stringData:
  application.properties: |
    #JDBC Configuration
    jdbc.driverClassName=com.mysql.jdbc.Driver
    jdbc.url=jdbc:mysql://db01.vprofile:3306/accounts?useUnicode=true&characterEncoding=UTF-
8&zeroDateTimeBehavior=convertToNull
    jdbc.username=admin
    jdbc.password=admin123
    #Memcached
    memcached.active.host=mc01.vprofile
    memcached.active.port=11211
    memcached.standBy.host=mc01.vprofile
    memcached.standBy.port=11211
    #RabbitMQ
    rabbitmq.address=rmq01.vprofile
    rabbitmq.port=5672
    rabbitmq.username=vprofile
    rabbitmq.password=guest
    #Elasticsearch (optional - not deployed)
    elasticsearch.host=vprosearch01
    elasticsearch.port=9300
    elasticsearch.cluster=vprofile
    elasticsearch.node=vprofilenode
---
# ================= Deployment =================
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vpro-app
  namespace: vprofile
spec:
  replicas: 2                          # the app tier is stateless ⇒ scale freely
  selector:
    matchLabels: { app: vpro-app }
  template:
    metadata:
      labels: { app: vpro-app }
    spec:
      # ---- wait for the backing services before Tomcat starts ----
      initContainers:
        - name: wait-db
          image: busybox:1.36
          command: ['sh','-c','until nc -z db01 3306;  do echo "waiting for db01";  sleep 3; done']
        - name: wait-mc
          image: busybox:1.36
          command: ['sh','-c','until nc -z mc01 11211; do echo "waiting for mc01";  sleep 3; done']
        - name: wait-rmq
          image: busybox:1.36
          command: ['sh','-c','until nc -z rmq01 5672; do echo "waiting for rmq01"; sleep 3; done']
      containers:
```

```
        - name: vpro-app
          image: alnaqib/vprofile-app:1.0
          ports:
            - containerPort: 8080
          # inject the real config over the (credential-free) file baked in the image
          volumeMounts:
            - name: app-config
              mountPath: /usr/local/tomcat/webapps/ROOT/WEB-INF/classes/application.properties
              subPath: application.properties
              readOnly: true
          readinessProbe:
            httpGet: { path: /, port: 8080 }
            initialDelaySeconds: 40
            periodSeconds: 10
          livenessProbe:
            tcpSocket: { port: 8080 }
            initialDelaySeconds: 60
            periodSeconds: 15
      volumes:
        - name: app-config
          secret:
            secretName: app-config
---
# ================= Service (NodePort → reached by HAProxy) =================
apiVersion: v1
kind: Service
metadata:
  name: app01
  namespace: vprofile
spec:
  type: NodePort
  selector: { app: vpro-app }
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30080              # every node exposes :30080 → app pods
```

WHY

Spring in VProfile (version 4.2) loads `classpath:application.properties`, which maps to `WEB-INF/classes/application.properties` inside the webapp. That is why we explode the WAR in the image — so that path exists as a real file — and the `subPath` mount replaces the empty file with the real one from the Secret. The result: **the exact same image** runs in dev and prod, the only difference being the injected Secret. That is what "a generic image with no secrets in it" means.

TIP

The real security win: previously the password was baked into the war — anyone pulling the image could read it. Now the image is completely clean and the password lives **only** in the `app-config` Secret — which is where RBAC and encryption-at-rest finally start to matter.

NOTE

The values in `app-config` (jdbc.password / rabbitmq.password) must match the ones in the `app-secret` that feeds the db and rmq containers. There are still two copies of the password, but **both are inside the cluster and protected** (not in an image). Next step if you want to remove the duplication: have the properties read `${{ENV}}` straight from app-secret.

3) Ordering & probes

WHY

The initContainers wait for `db01`, `mc01`, and `rmq01` to be up (via `nc -z`) before Tomcat starts. Without them, the app could start and try to reach a database that is not ready yet and end in `CrashLoopBackOff`. That is what "ordering" means in microservices — the dependency is resolved deliberately, not by luck.

TIP

The `readinessProbe` keeps HAProxy from sending traffic to a pod that is still warming up (Tomcat takes time to boot). And `replicas: 2` because this tier is stateless, so we can spread the load.

BUILD · SCAN · PUSH

Build · Scan · Push

08

Now we build the four images, scan them with Trivy, and push them to `alnaqib/*`.

build-and-push.sh — BASH

```
#!/usr/bin/env bash
# Build, scan and push the 4 VProfile images to Docker Hub (alnaqib/*)
# Run from a workstation that has: git, docker (or podman), trivy
set -euo pipefail

DHUB="alnaqib"        # your Docker Hub username
TAG="1.0"
PLATFORM="linux/amd64"   # RHEL 9 nodes are amd64

# 0) get the source repo (app + db images build FROM it)
[ -d src ] || git clone -b Master https://github.com/abdelrahmanonline4/sourcecodeseniorwr.git src

# 1) build
docker build --platform "$PLATFORM" -t "$DHUB/vprofile-app:$TAG" -f images/app/Dockerfile       src/
docker build --platform "$PLATFORM" -t "$DHUB/vprofile-db:$TAG"  -f images/db/Dockerfile        src/
docker build --platform "$PLATFORM" -t "$DHUB/vprofile-mc:$TAG"  -f images/memcached/Dockerfile
images/memcached/
docker build --platform "$PLATFORM" -t "$DHUB/vprofile-rmq:$TAG" -f images/rabbitmq/Dockerfile
images/rabbitmq/

# 2) scan (fail the build on HIGH/CRITICAL if you drop '|| true')
for s in app db mc rmq; do
  echo "== scanning $s =="
  trivy image --severity HIGH,CRITICAL --ignore-unfixed "$DHUB/vprofile-$s:$TAG" || true
done

# 3) push
docker login -u "$DHUB"
for s in app db mc rmq; do docker push "$DHUB/vprofile-$s:$TAG"; done
echo "done."
```

WHY

The app image now has **no password** — so there is no secret `--build-arg` at all, and you do not rebuild it when a password changes. The change happens only in the `app-config` Secret plus a `rollout restart` of the deployment. Note also `--platform linux/amd64` since all nodes are RHEL 9 / amd64.

NOTE

If you keep the repos **private** on Docker Hub, the nodes cannot pull the images without an `imagePullSecret`. Create it like this:
`kubectl -n vprofile create secret docker-registry regcred --docker-username=alnaqib --docker-password=...`
then add `imagePullSecrets: [name: regcred]` to the pod spec.

TIP

`trivy` stops at `HIGH/CRITICAL`. If you want it to actually fail the build (instead of continuing), drop the `|| true` from its line.

DEPLOY IN ORDER

Deploy in order

09

Order matters: namespace ← secret ← backing services (db, mc, rmq) ← the app. The initContainers protect us if there is a small overlap, but the cleanest path is to go in order.

deploy.sh — BASH

```
#!/usr/bin/env bash
# Deploy VProfile to the cluster (run where kubectl is configured, e.g. master1)
set -euo pipefail

# 1) storage class (once per cluster)
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.31/deploy/local-path-
storage.yaml
kubectl -n local-path-storage rollout status deploy/local-path-provisioner
kubectl patch storageclass local-path \
  -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'

# 2) app stack (order matters: ns → secret → backing services → app)
kubectl apply -f k8s/00-namespace.yaml
kubectl apply -f k8s/01-secret.yaml
kubectl apply -f k8s/02-db.yaml
kubectl apply -f k8s/03-memcached.yaml
kubectl apply -f k8s/04-rabbitmq.yaml
kubectl apply -f k8s/05-app.yaml

# 3) watch it come up
kubectl -n vprofile get pods -w
```

Watch it come up

on master1 — BASH

```
$ kubectl -n vprofile get pods
NAME                READY   STATUS      RESTARTS
vpro-db-xxxx        1/1     Running     0
vpro-mc-xxxx        1/1     Running     0
vpro-rmq-xxxx       1/1     Running     0
vpro-app-xxxx       0/1     Init:2/3    0     # still waiting for rmq
vpro-app-xxxx       1/1     Running     0     # ready
```

NOTE

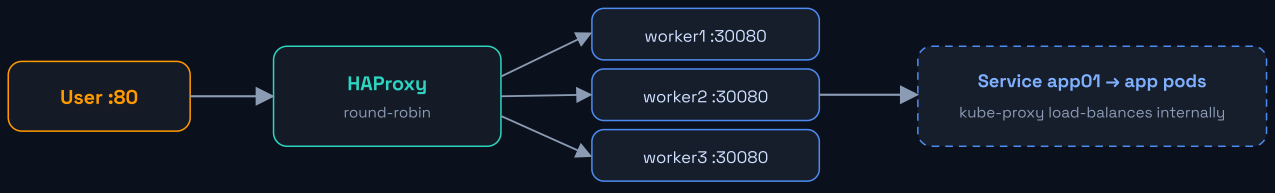
On the first run, the db image seeds the tables from `db_backup.sql` — this may take a few seconds. The app stays in `Init` until db/mc/rmq answer.

EXPOSE VIA HAPROXY

Expose via HAProxy

10

The `app01` Service is a NodePort — every node in the cluster opens port `30080` and forwards it to the app pods. HAProxy load-balances requests across the nodes.



User → HAProxy(round-robin) → any node :30080 → kube-proxy → app pod

Your current config (k8s-api)

Your HAProxy already load-balances the Kubernetes API across the 3 masters. We build on that — adding a new frontend/backend for the app, without touching what already works.

/etc/haproxy/haproxy.cfg (current) HAPROXY

```
#---------------------------------------------------------------------
# GLOBAL
#---------------------------------------------------------------------
global
    log /dev/log local0
    log /dev/log local1 notice
    maxconn 4000
    user  haproxy
    group haproxy
    daemon
#---------------------------------------------------------------------
# DEFAULTS   (note: mode tcp is the default here)
#---------------------------------------------------------------------
defaults
    log     global
    mode    tcp
    option  tcplog
    option  dontlognull
    timeout connect 10s
    timeout client  86400s
    timeout server  86400s
    timeout tunnel  86400s
    retries 3
#---------------------------------------------------------------------
# FRONTEND - Kubernetes API
#---------------------------------------------------------------------
frontend k8s-api
    bind *:6443
    mode tcp
    default_backend k8s-masters
#---------------------------------------------------------------------
# BACKEND - Kubernetes Control Plane Nodes
#---------------------------------------------------------------------
backend k8s-masters
    mode tcp
    balance roundrobin
    option tcp-check
    server master1 10.0.12.138:6443 check inter 2s fall 3 rise 2
```

```
    server master2 10.0.12.150:6443 check inter 2s fall 3 rise 2
    server master3 10.0.12.186:6443 check inter 2s fall 3 rise 2
```

GOTCHA

Note that your `defaults` use `mode tcp` (correct for `k8s-api` on 6443). So the app frontend/backend must set `mode http` explicitly — which the config below does.

1) Get the node IPs

on master1 — BASH

```
$ kubectl get nodes -o wide
# take the INTERNAL-IP column for worker1/2/3
```

2) Append the app config to HAProxy

/etc/haproxy/haproxy.cfg (append) — HAPROXY

```
# ------------------------------------------------------------
#  Add to /etc/haproxy/haproxy.cfg  (or /etc/haproxy/conf.d/)
#  on the existing HAProxy VM, then:  systemctl reload haproxy
#  Replace the IPs with:  kubectl get nodes -o wide  (INTERNAL-IP)
# ------------------------------------------------------------
frontend vprofile_http
    bind *:80
    mode http
    default_backend vprofile_app_nodes

backend vprofile_app_nodes
    mode http
    balance roundrobin
    option httpchk GET /login
    http-check expect status 200
    server worker1 10.0.x.x:30080 check
    server worker2 10.0.x.x:30080 check
    server worker3 10.0.x.x:30080 check
```

3) Validate & reload

on HAProxy VM — BASH

```
# validate the config first
$ sudo haproxy -c -f /etc/haproxy/haproxy.cfg
Configuration file is valid

# reload with no downtime
$ sudo systemctl reload haproxy
```

WHY

`option httpchk GET /login` makes HAProxy health-check each node. It matters that we check `/login` and not `/`: the app answers `GET /` with a `302 redirect` to the login page, so if we checked `/` expecting status 200, HAProxy would mark every node **DOWN** and return `503`. The `/login` page returns `200` directly, so the check passes.

TIP

The NodePort is open on **every** node (masters too), so in theory you can point at any nodes. We chose the workers since they normally run the workloads.

VERIFY & TROUBLESHOOT

Verify & troubleshoot

11

Confirm everything works, and get ready for the most common issues.

on master1 BASH

```
$ kubectl -n vprofile get all
$ kubectl -n vprofile get pvc,secret

# logs of any service
$ kubectl -n vprofile logs deploy/vpro-app
$ kubectl -n vprofile logs deploy/vpro-db

# describe a pod if something is wrong
$ kubectl -n vprofile describe pod <pod-name>

# quick in-cluster connectivity test
$ kubectl -n vprofile run t --rm -it --image=busybox:1.36 -- \
    sh -c "nc -z db01 3306 && echo db-ok"
```

Quick failure table

Symptom	Likely cause	Diagnose / fix
Pod in **ImagePullBackOff**	wrong image name/tag or private repo	`kubectl describe pod ...` ; check `alnaqib/vprofile-*:1.0` and `regcred`
PVC **Pending**	no default StorageClass	`kubectl get sc` ; go back to part 03
vpro-db CrashLoop	old PVC with data on a different password	delete the PVC and redeploy (loses data) or fix the Secret
app stuck in **Init**	db/mc/rmq not ready	`kubectl -n vprofile get pods` ; logs of the missing service
HAProxy returns **503**	wrong NodePort or nodes down	`curl http://NODE_IP:30080` from the HAProxy VM
page opens but **login** fails	app-config password ≠ DB Secret password	match `jdbc.password` in app-config with `db-password` in app-secret

TIP

Once everything is `Running`, open `http://HAPROXY_IP/` in the browser — you should see the VProfile login page. Try registering a user; if it works, the DB is up, and if the page renders cleanly, the cache and broker connections are fine too.

WRAP-UP

Wrap-up & next steps

12

We turned **VProfile** from 5 VMs into **4 microservices** on a real HA cluster: we built hardened images ourselves, wired the secrets correctly, ordered the dependencies with initContainers, persisted the DB state, and opened the door to the world through HAProxy.

What we learned (the concepts)

- The Service name = the DNS name — that is why service names are not random.
- A multi-stage build separates build tools from the runtime = a smaller, safer image.
- Images are **credential-free**: secrets are injected at runtime from a Secret (env or mount), not baked into image layers.
- Dependency between services is solved with initContainers, not luck.
- State needs a PVC; a cache does not.

Next steps (production-grade)

- **Ingress + TLS** instead of NodePort (nginx-ingress / cert-manager) if you want HTTPS inside the cluster.
- **Remove the password duplication**: have `app-config` read `${ENV}` from app-secret (Spring 4.2 supports it) instead of a second copy — a single source of truth.
- **Encryption at rest** on etcd + **RBAC** to limit who can read secrets — now meaningful because the image is clean.
- **HPA**: autoscale the app tier by load.
- **Secrets manager** (Vault / Sealed Secrets) instead of a plain Secret.
- **Monitoring**: Prometheus + Grafana to watch the four services.

> TIP
> All files (Dockerfiles + manifests + scripts) are in the `bundle` shipped alongside this guide — usable directly once you edit the IPs and passwords.

ALnaqib

Yousef Salem • DevOps • Kubernetes • Cairo • 2026